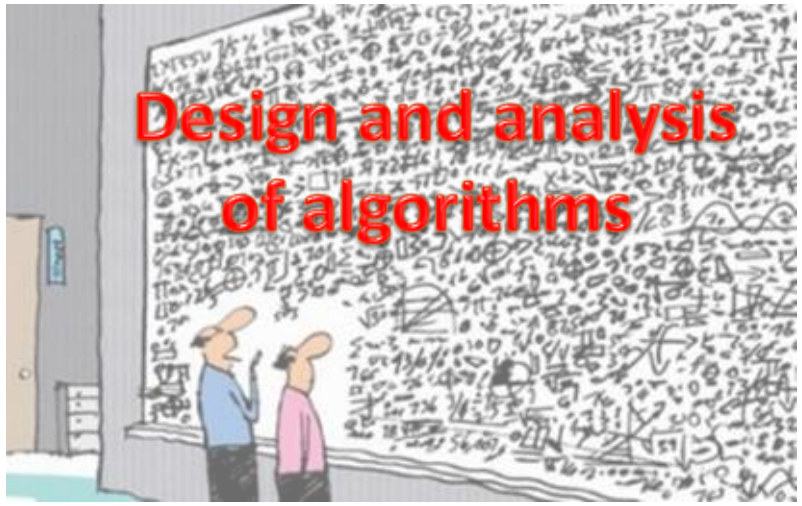


- Shortest paths: III
  - Extensions to Dijkstra's algorithm
  - A*

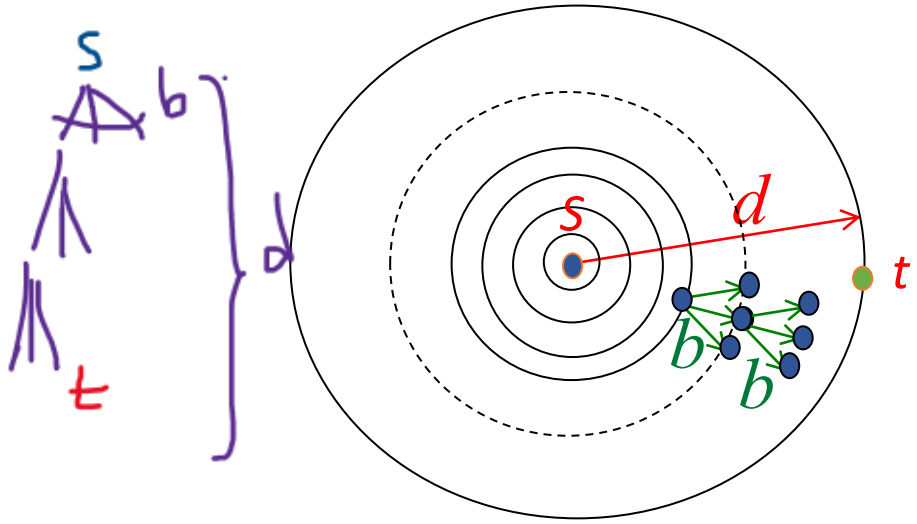
Objective

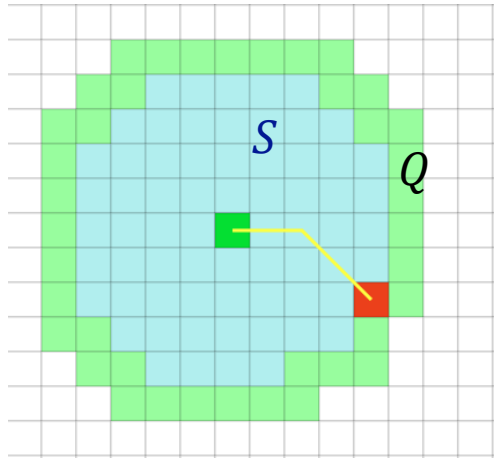
- Dijkstra's algorithm explores shortest paths to all nodes.
- Focus on finding the shortest path to a particular destination.
- Use heuristics to make our search more efficient.
- Read (if you have plenty of free time) "Speed-Up Techniques for Shortest-Path Computations", by Dorothea Wagner and Thomas Willhalm
- https://www.youtube.com/watch?v=cSxnOm5aceA.
- For A* play with: https://qiao.github.io/PathFinding.js/visual/.

Speeding up Dijkstra's algorithm

- Most of the searches: single source s to single destination t.
- Obvious: stop the search when t is removed from Q.
- Simple model: assume a b-branching structure, all edges of length 1.

Dijkstra explores all possible directions in increasing distance from s

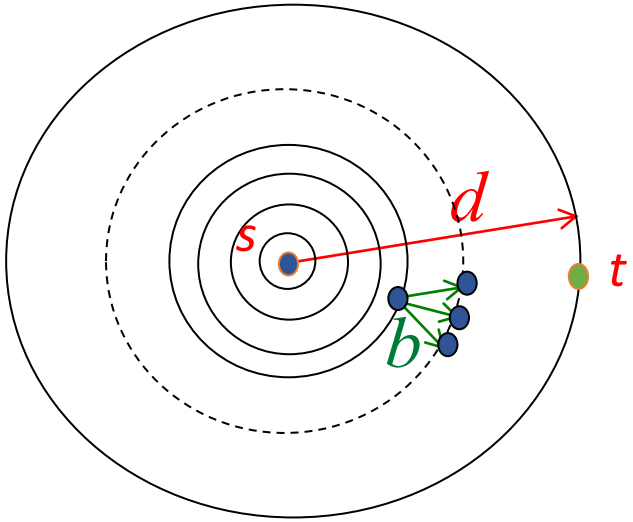


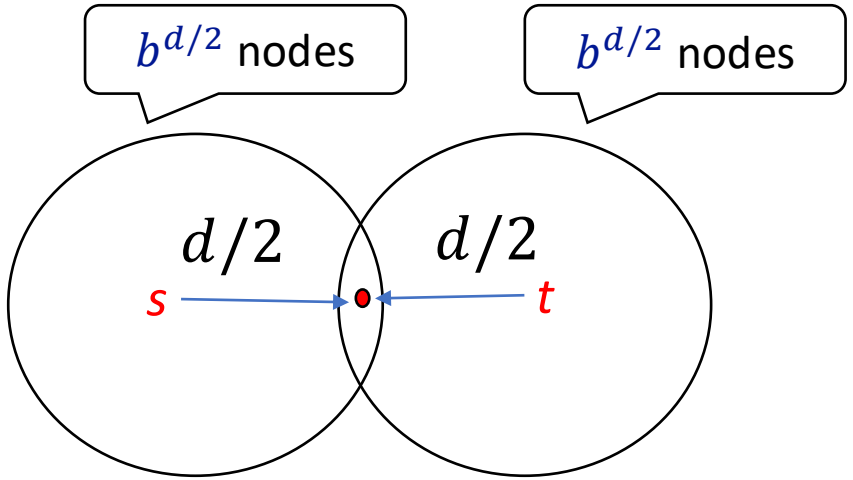


dark green: s
red: t
green: in set Q, i.e., partially explored,
blue: already explored.

- To find the shortest path to a node t that is at distance d away (with one-directional search) we need to visit about $n = b^d$ nodes.

Bi-directional search





Normal Dijkstra explores $n = b^d$ nodes

Bidirectional search explores $n_{BD} = 2b^{d/2} = 2\sqrt{b^d} = 2\sqrt{n}$

- With bi-directional search we explore two graphs of diameter $\frac{d}{2}$, hence explore about $O(\sqrt{n})$ nodes.

in this specific type of graph

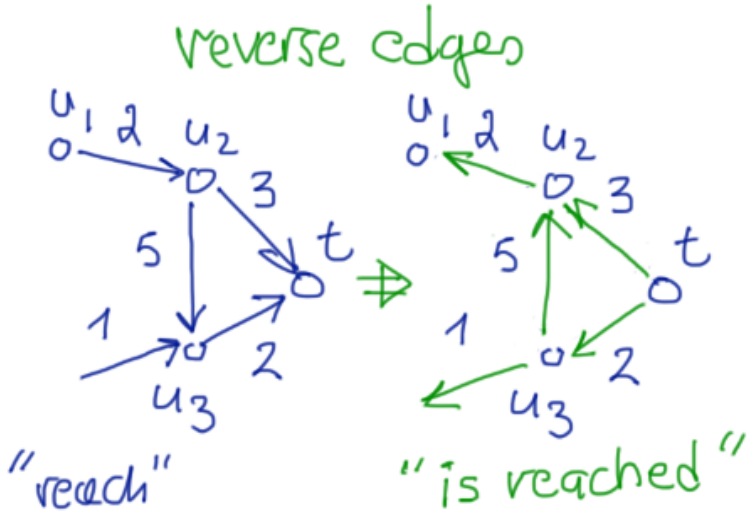
Bi-directional search

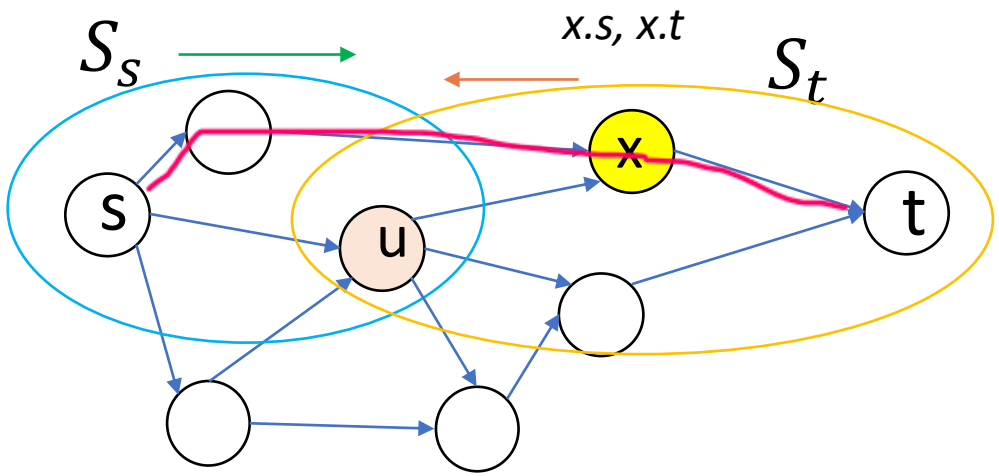
estimated distance in each search.

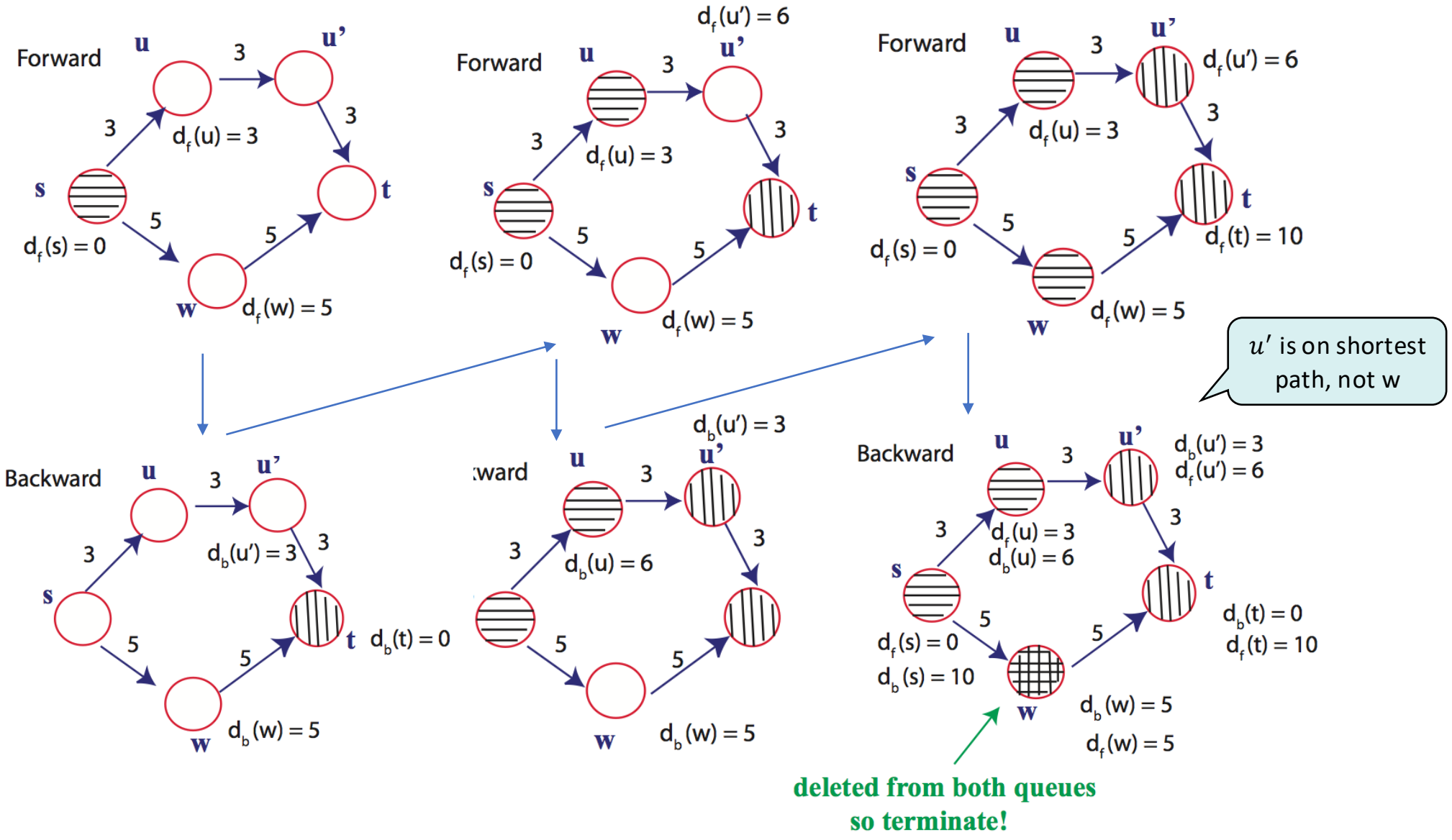
Bidirectional search: perform *in parallel*:

- Search forward from *s* with Dijkstra (S_s : decided nodes, keep *v.s*).
- Search backwards from *t* with Dijkstra (S_t: decided nodes, keep *v.t*).
- Stop when some node *u* has been placed in both S_s and S_t.
- Check for node *x in* S_s with minimum value of *x.s* + *x.t*.

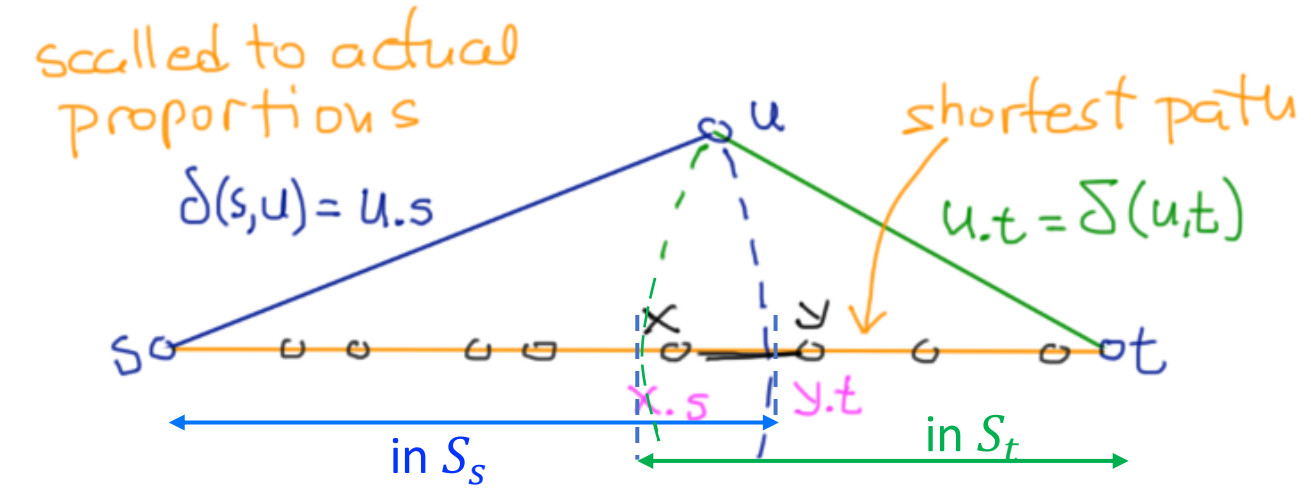
this node is on shortest path *s* - *t*.






Forward
u
u'
3
3
3
$d_f(u) = 3$
s
t
5
5
$d_f(s) = 0$
w
$d_f(w) = 5$
$d_f(u') = 6$
$d_f(t) = 10$
Backward
$d_b(u') = 3$
$d_b(t) = 0$
$d_b(w) = 5$
$d_b(u) = 6$
$d_b(s) = 10$
$d_f(s) = 0$
$d_f(w) = 5$
$d_b(u') = 3$
$d_f(u') = 6$
$d_f(u) = 3$
$d_b(u) = 6$
$d_b(t) = 0$
$d_f(t) = 10$
$d_b(w) = 5$
$d_f(w) = 5$
u' is on shortest path, not w
deleted from both queues so terminate!

Proof (not in material)



shortest s-t path must use only nodes in $S_s \cup S_t$

x: node on shortest path with largest x.s < u.s

$\Rightarrow x \in S_s$ (since x.s < u.s)

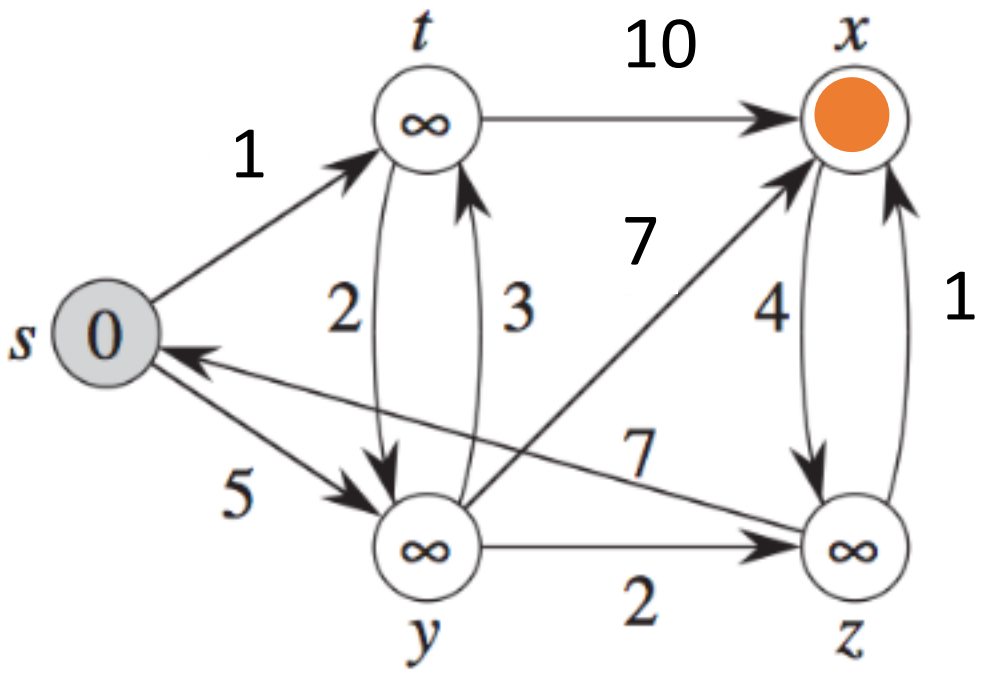
y: next node on shortest path $\Rightarrow y \in S_t$ (since $y.t \leq u.t$)

Since (x,y) has been relaxed $\Rightarrow x.t = y.t + w(x,y) = \delta(x,t)$

$\Rightarrow x.s + x.t = \delta(s,x) + \delta(x,t) = \delta(s,t)$ = minimal among all nodes.

Exercise

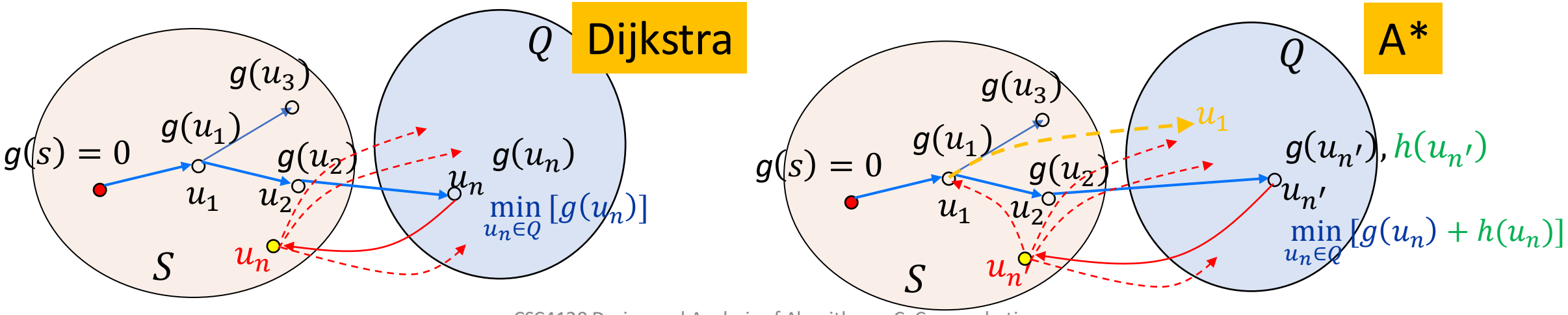
- Run bi-directional Dijkstra to find the shortest path from *s* to to *x*.



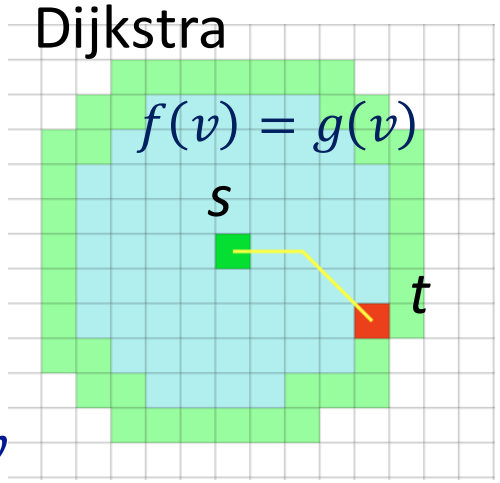
A* with potential function $h(.)$

- Keeps a closed set S, open set Q, **Stop** when you bring destination t in S.
- **Priority rule**: bring in S the node u_n from Q that has $\min_{u_n \in Q} [g(u_n) + h(u_n)]$.
- **Relaxations**: update $g(.)$ for all successors of v, including nodes in S.
  - $g(v)$: shortest distance to s *through nodes in* S.
- All the "strictly" updated successors of v are put back to Q even if they were in S.

priority function $f(u)$

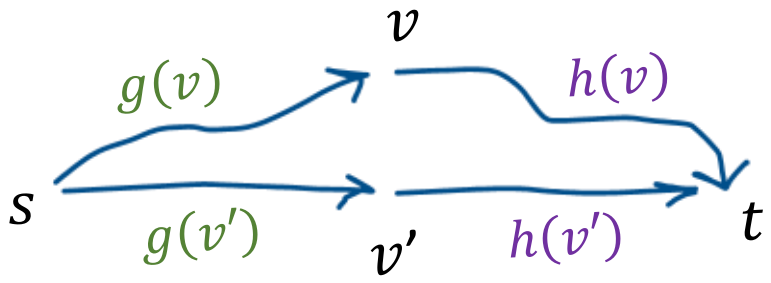


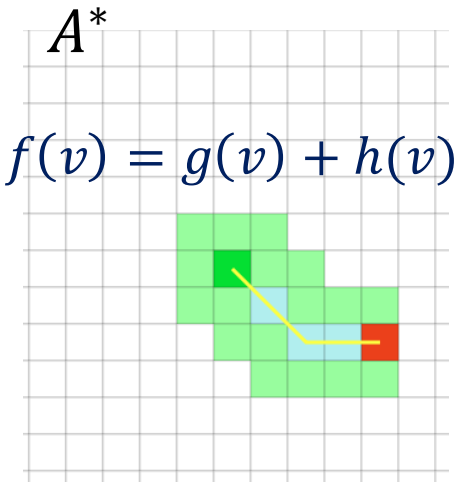
Potential function $h(v)$



- Let $h(v)$ = *estimate of future cost to reach t*: min cost $v \to t$.
- Then $f(v) = g(v) + h(v)$ = min cost from s to reach t **through v**.
- We will move along the shortest path, since $f(v)$ is minimized for v on shortest $s \to t$ path.

$$\min_v \{g(v) + h(v)\}$$



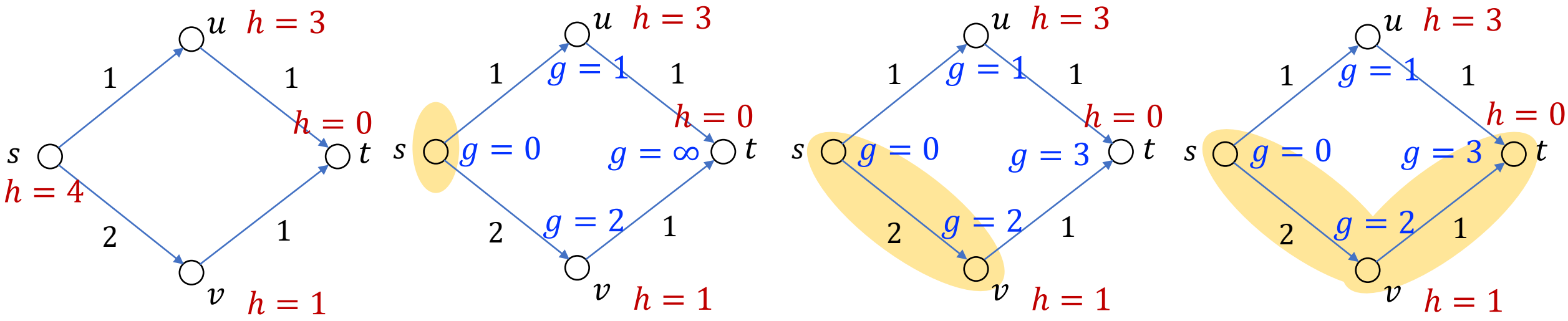


- A^*: use $h(v)$ = heuristic that approximates $\delta(v, t)$.

$h(.)$ is a "potential function. It decreases when we get closer to t.

Things can go wrong

- Run $A*$ on the following graph and check that it will terminate with the wrong shortest path to t. Use $h = 4,3,1,0$ for s, u, v, t.



- Can't happen if h is "admissible": need $h(v) \leq \delta(v, t)$. $h(u) = 3 > \delta(u, t) = 1$
- Other issues? nodes may go in and out of S: no guarantee on termination time. To avoid that, we need h to be "consistent". Consistency implies admissibility.

Desired properties of potential function $h(.)$

- New issues to worry about:
  a) when the target t enters S, do we have a shortest path?
  b) nodes in S can move back to Q?

correctness

need bound on # of steps

- If $h(.)$ satisfies certain properties, then I don't not have to worry!
  - **Admissible** $h(.)$: when the target t enters S, we have a shortest path.
    - But no upper bound on max # of steps.
    - Requires $h(v) \leq \delta(v, t)$.
  - **Consistent** $h(.)$: works like normal Dijkstra
    - Requires $w_{ij} + h(u_j) - h(u_i) \geq 0$.
    - Nodes in S never move back to Q, when target t enters S, we have a shortest path.

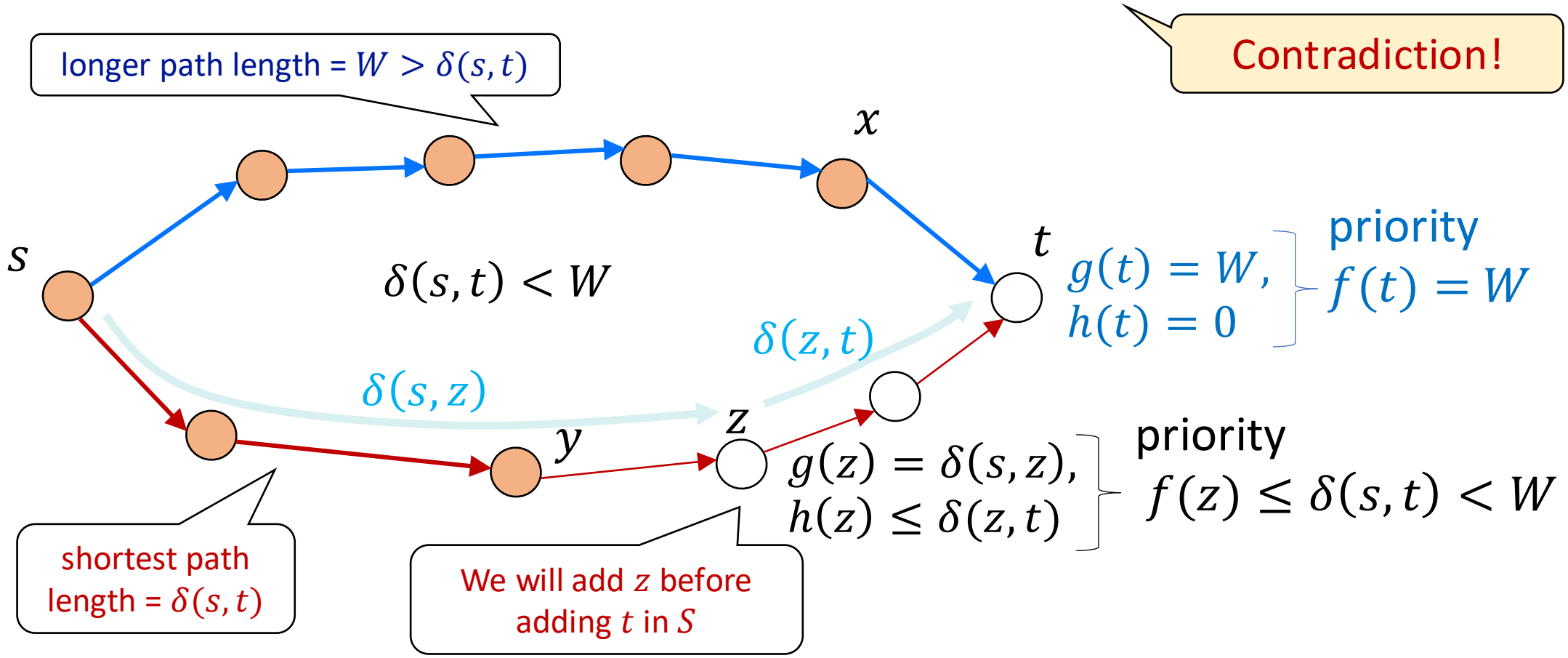
this is a **typical case** for A*: correct but no performance guarantee.

explores shortest path tree

- Consistency ⇒ admissibility.

Proof of property of admissible heuristics

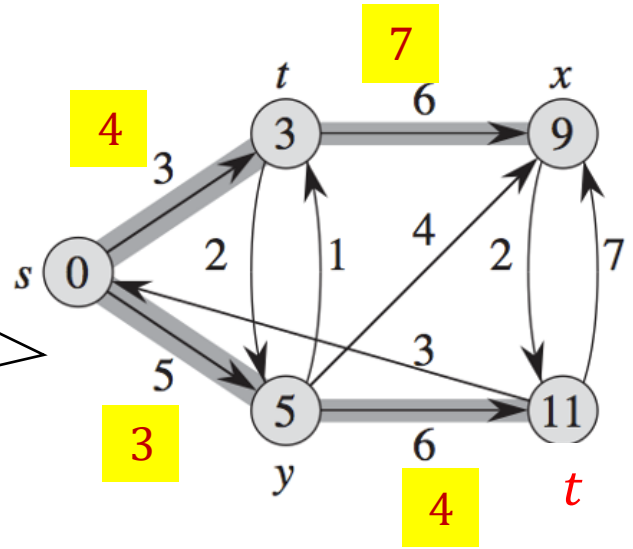
- Admissible $h(.)$: $\forall z, h(z) \leq \delta(z,t)$.
- Suppose that t will enter S along the blue path, before some z along the red SP(s,t).



Consistent potentials: Dijkstra with w'

since $h(.)$ consistent

- Let's run Dijkstra with $w'(u,v) = w(u,v) + h(v) - h(u) \geq 0$.
- Visits the *same SPT* (according to old weights $w(u,v)$, see next slide), but in a different order that reflects the information of $h(.)$.
  - If $h(v) \downarrow$ as v gets closer to the goal t, then *edges include future costs*: An edge *becomes shorter* if it brings us closer to t.
  - In this new SPT based on w', the goal t is *closer to the origin*.
  - It will be lifted earlier from the table!
  - Keeps all the good properties!

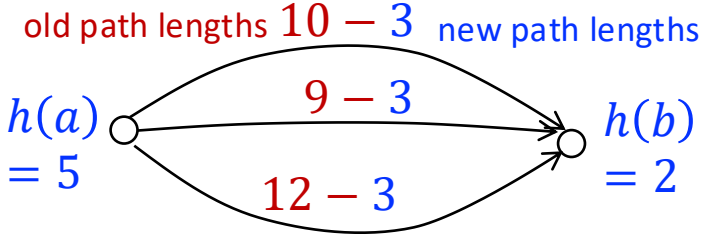


The SPT remains the same but t comes closer to s while other parts far from t are further away

Modifying weights using h: same shortest paths

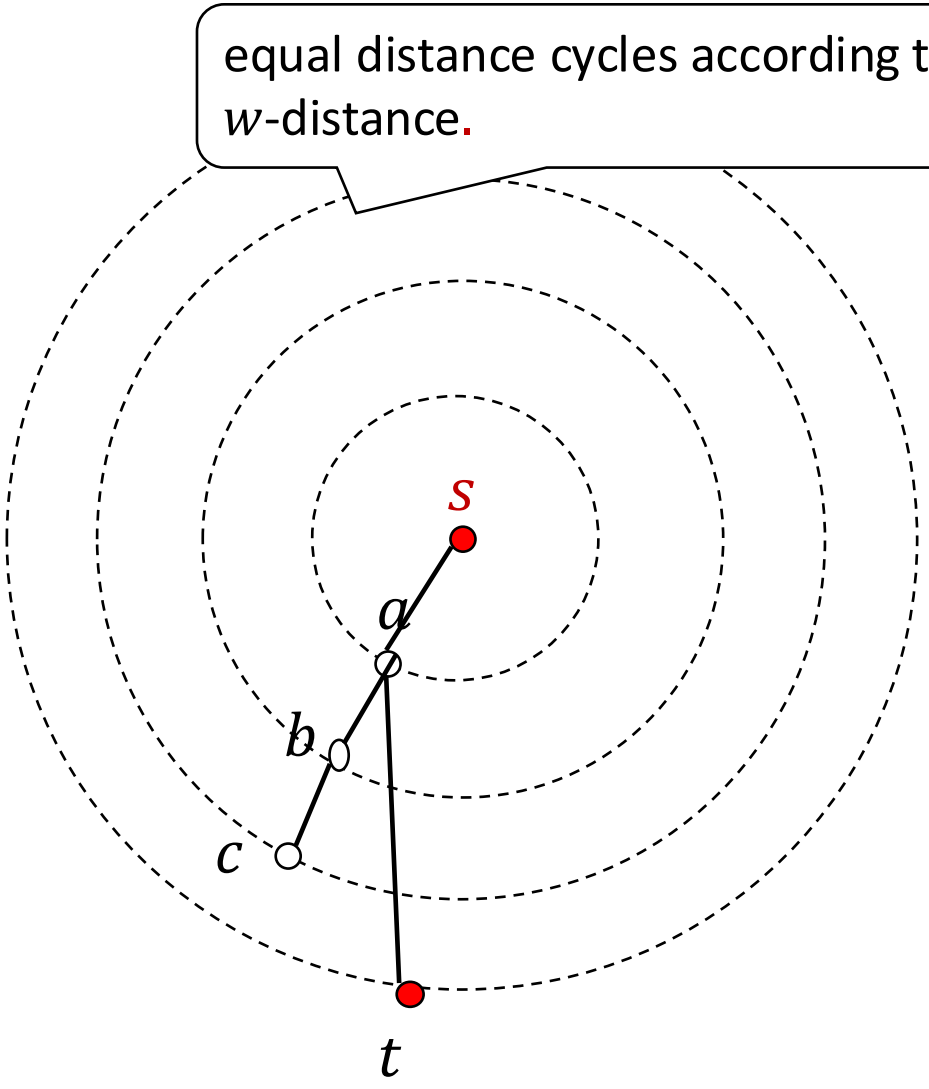
- Suppose $w'(u,v) = w(u,v) + h(v) - h(u)$.
- **Property**: the shortest paths between any pair of nodes are the same under w, w'.
  - The length of any $a - b$ path with modified weights is the same up to a constant that depends on a, b.

$$w'(a = u_1, u_2, \dots, u_n = b) = \sum_{i=1}^{n-1} w(u_i, u_{i+1}) - \sum_{i=1}^{n-1} (h(u_i) - h(u_{i+1}))$$
$$= \sum_{i=1}^{n-1} w(u_i, u_{i+1}) + h(b) - h(a).$$

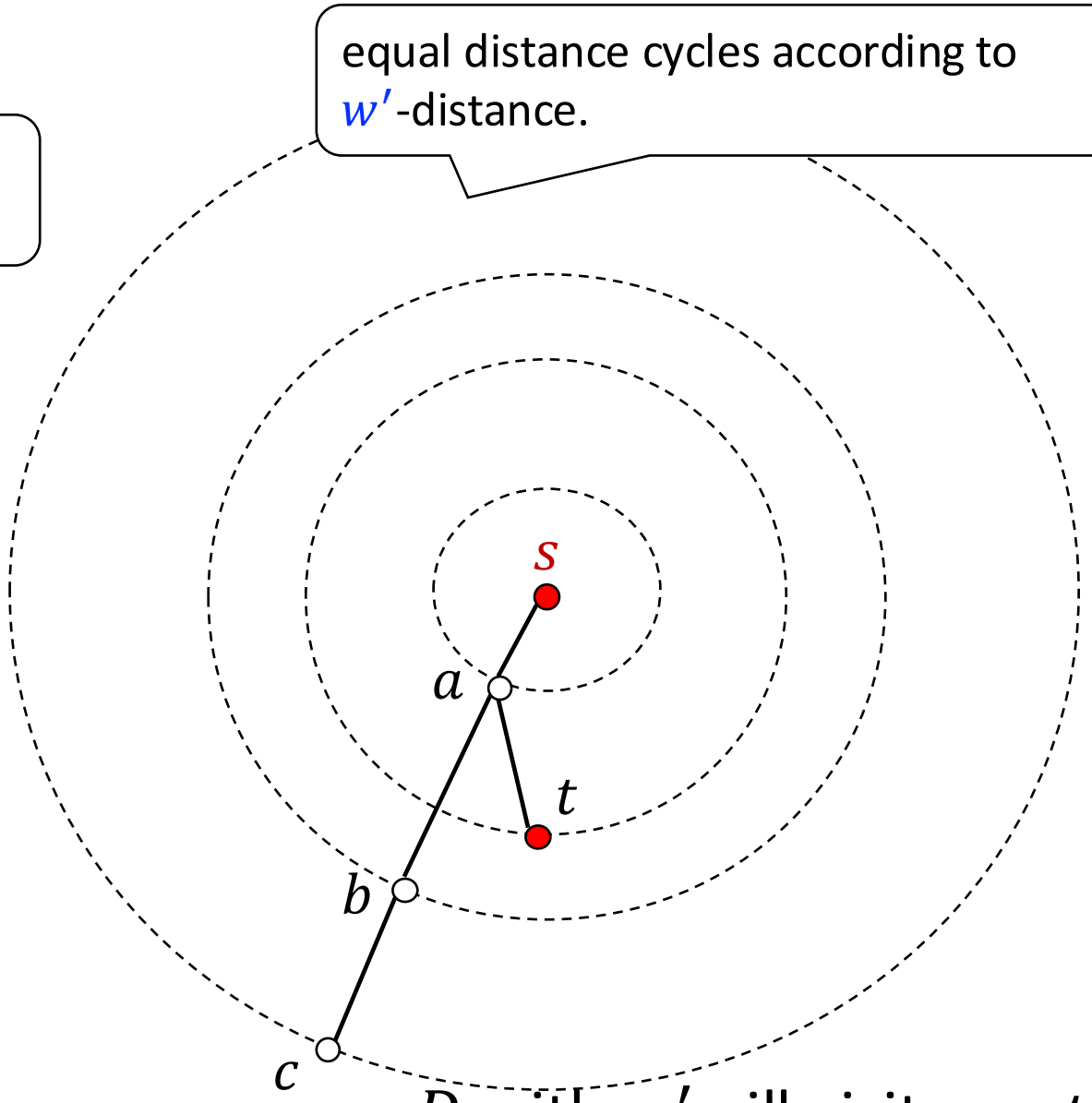


- **Corollary:** Although the weights w' are different, the shortest paths from s don't change, and the SPTs rooted at s are the same under w, w'.

SPTs



D with w will visit s, a, b, c, t



D with w' will visit s, a, t

Dijkstra with w' is A^*

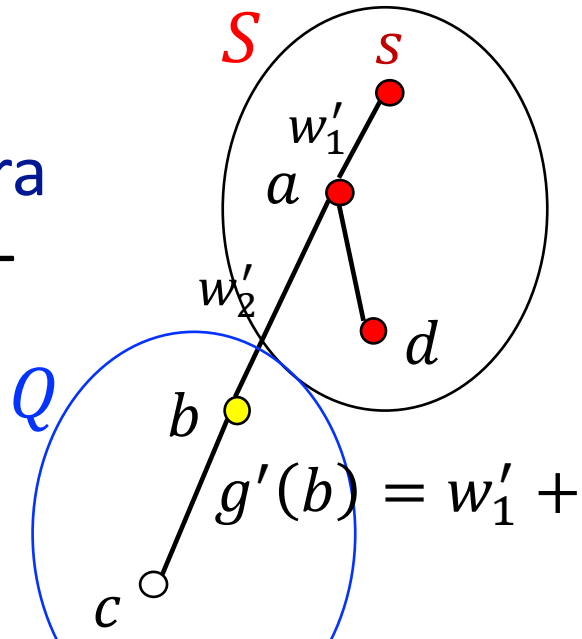
- **Theorem**: Dijkstra with $w'(u,v) = w(u,v) + h(v) - h(u) \geq 0$ does the same steps as A^* with $h(.)$.
- A^* is essentially Dijkstra with new edge weights to reflect future costs.

Proof

- Suppose at time t, both Dijkstra with w', and A^* with $w, h(.)$, constructed the same closed sets. We prove that in the next step they will bring into S the same node from Q.
  - Which is the next node according to the w'-SPT.

Dijkstra

w'-SPT



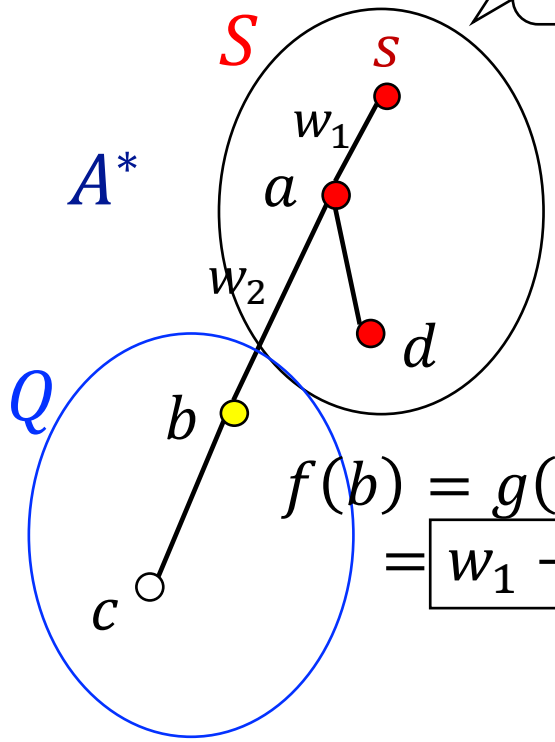
$g'(b) = w_1' + w_2'$

$= w_1 + h(a) - h(s) + w_2 + h(b) - h(a)$

$= \boxed{w_1 + w_2 + h(b)} - h(s)$

It has formed the w-SPT with nodes in S, same as w'-SPT

A^*



$f(b) = g(b) + h(b)$

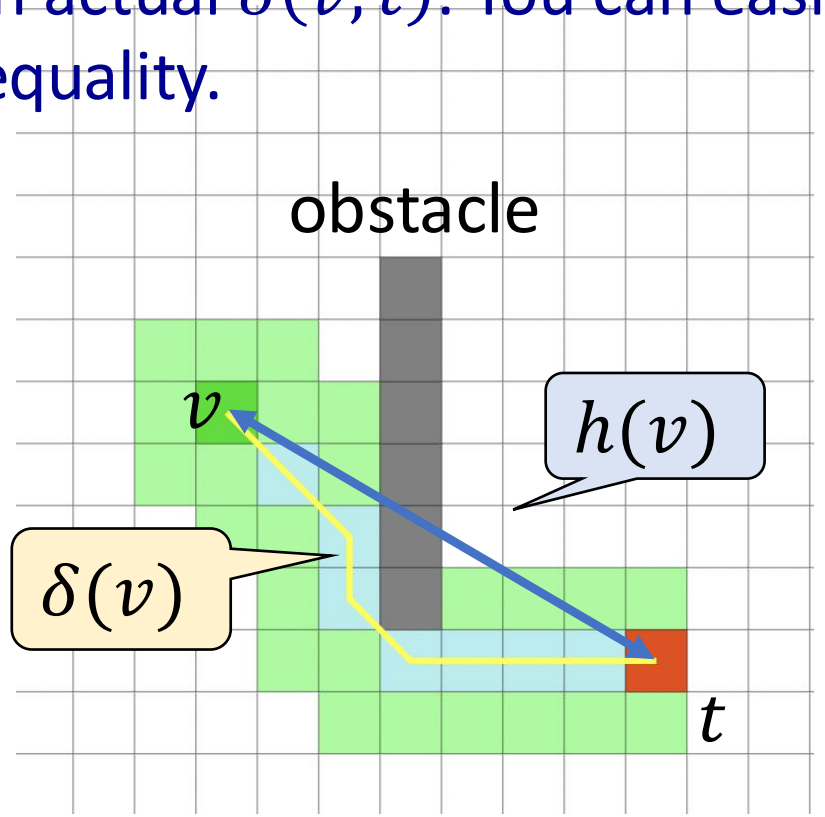
$= \boxed{w_1 + w_2 + h(b)}$

Practical ways to define $h(.)$

- In the next slides we show some practical methodologies for using potential functions for A^*.
- The first is valid for Euclidean graphs, i.e., graphs that connect points on an actual map.
- The second (landmarks) is more general and makes no assumptions on the graphs.
- In each case we need to show that the function $h(v)$ we construct is consistent, i.e., $w(u,v) + h(v) - h(u) \geq 0$.

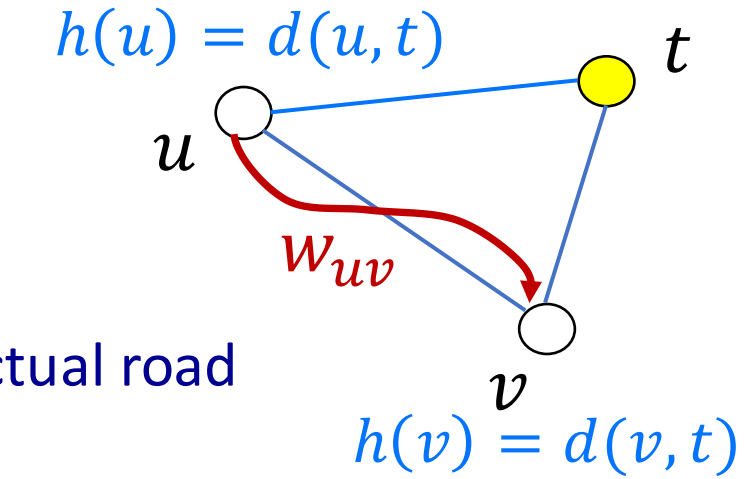
Euclidian space with obstacles

- Typical case for moving in a Euclidian space with obstacles.
- $h(v) =$ straight distance $d(v, t)$. Can be measured on the map. It is always a lower bound on actual $\delta(v, t)$. You can easily prove it is also consistent by using the triangle inequality.



$h(v) \leq \delta(v)$

Euclidian graphs



- Find shortest path to t in **geographical maps**.
  - Graph: nodes are locations, edges are roads, weights are actual road distances.
  - Use as $h(u)$ the (straight) Euclidean distance $d(u,t)$ to target t.
  - $w_{uv} + h(v) - h(u) \geq 0$? $w_{uv} \geq d(u,v) \geq \underbrace{d(u,t)}_{h(u)} - \underbrace{d(v,t)}_{h(v)}$.

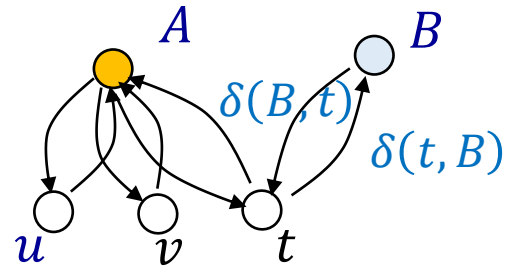
- Find shortest path to t in **arbitrary graphs** (V,E) (see next slide)
  - Prepick a subset of nodes $L \subset V$, the "**landmarks**".
  - Precompute the shortest distances $\delta(u,l), \delta(l,u)$ for all $u \in V, l \in L$.
  - Use as $h(u) = \max_{l \in L}\{\max\{\delta(u,l) - \delta(t,l), \delta(l,t) - \delta(l,u)\}\}$.

compute this for each landmark l

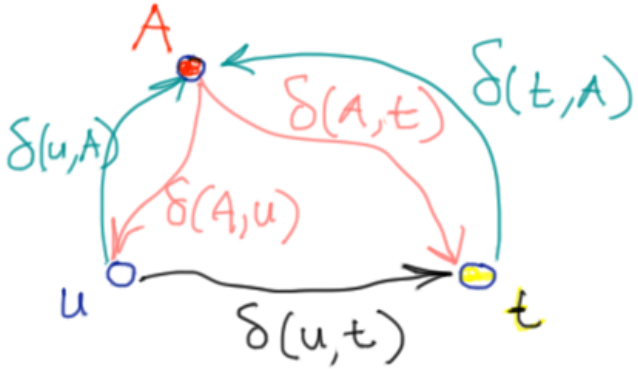
choose the max over all landmarks

why is this consistent? needs proof

General graphs: Landmarks



- Choose as $h_t(u)$ an estimate of $\delta(u,t)$ based on "landmarks".
- For each specific landmark node $A, B, \ldots$ pre-compute SP distances to and from all nodes of the graph.
- Get "good" lower bounds on actual distance $\delta(u,t)$, use it as $h_t(u)$.
- Is more general, works for general graphs, no Euclidian distance involved.
- $\delta(u,t) + \delta(t,A) \geq \delta(u,A) \Rightarrow \delta(u,t) \geq \delta(u,A) - \delta(t,A)$
- $\delta(A,u) + \delta(u,t) \geq \delta(A,t) \Rightarrow \delta(u,t) \geq \delta(A,t) - \delta(A,u)$
- Hence, $\delta(u,t) \geq \max\{\delta(u,A) - \delta(t,A), \delta(A,t) - \delta(A,u)\} \triangleq L(A,u,t)$.



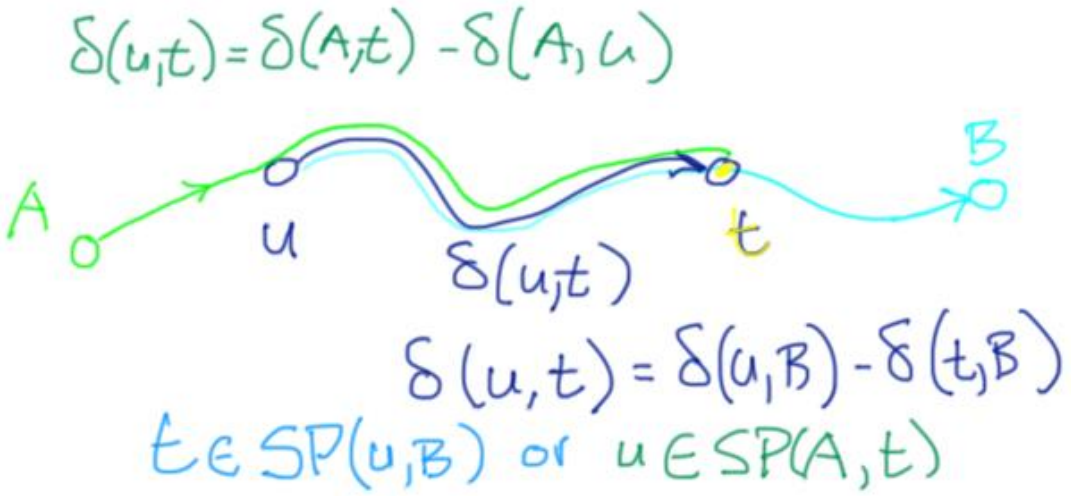
$$\delta(u,t) \geq \max\{L(A,u,t),\ L(B,u,t), L(C,u,t), \ldots\}$$

$$\triangleq h_t(u)$$

This proves admissibility of $h_t(u)$. Can also prove that it is consistent (in HW)

Landmarks (cont.)

- When are the lower bounds tight? $h_t(u) = \delta(u, t)$.
- A "best" landmark X: u is on $SP(X, t)$ or t is in $SP(u, X)$.
  - In this case the lower bounds are tight.



Conclusions

- When we care for the SP to a single destination Dijkstra may be inefficient since it explores the graph towards all possible directions.
- There are some smart ways to remedy that. We examined bi-directional search, and potential-based methods. These later ones modify Dijkstra's algorithm to focus its search towards the destination.
- A^* is an AI algorithm that finds shortest paths on the plane when obstacles are inserted. It uses a potential function h that approximates the future costs from any given node.
- A^* can be seen as a specific case of Dijkstra's algorithm with weights that reflect future costs.